PR Outreach Intelligence — Testing Guide

Version: 1.0
Date: April 2026
Stack: Express 5 · Vue 3 · Vitest · PostgreSQL · pnpm monorepo
Total Tests: 206 (179 backend + 27 frontend)
Status: All passing ✓

---

1. Why We Test (And Why It Matters Here)

PR Outreach Intelligence isn't a todo app. It handles **authentication with token rotation**, **transactional email sending**, **AI-generated content**, and **multi-table PostgreSQL transactions**. A single broken query can corrupt relationship scores across contacts, campaigns, and interactions.

Testing this project serves three purposes:

1. **Catch regressions before they reach production.** When you change the interaction scoring logic, you need to know immediately if campaigns break.
2. **Document behavior as executable specs.** Reading `auth.service.test.ts` tells you more about token rotation than any wiki page.
3. **Enable confident refactoring.** The codebase uses raw SQL everywhere (no ORM safety net). Tests are the safety net.

> The rule of thumb: if it touches the database, handles money, or manages auth, it gets tested. No exceptions.

---

2. The Stack at a Glance

Layer	Technology	Test Tool
Backend runtime	Express 5.2 + TypeScript 6	Vitest 4.1 (node environment)
Database	PostgreSQL 15 (raw `pg`, no ORM)	Real test DB (`pr_ai_db_test`)
Validation	Zod 4.3	Unit tests (schema `.safeParse()`)

Auth	JWT + bcryptjs + SHA-256 refresh tokens	Integration tests against real DB
AI providers	OpenAI / Gemini via native fetch	Mocked with `vi.mock()`
Email	Resend / Console provider	Mocked with `vi.mock()`
Frontend framework	Vue 3.5 + Pinia 3.0 + Vue Router 5	Vitest 4.1 (jsdom environment)
Frontend HTTP	Native fetch (no Axios)	`vi.fn()` stubs
Component testing	@vue/test-utils 2.4	jsdom
Monorepo	pnpm workspaces	Shared `@pr-outreach/shared-types`

---

3. Project Structure (Test Files)

```
backend/src/
├── __tests__/
|   ├── setup.ts              ← env bootstrap
|   └── helpers.ts            ← cleanDatabase(), createTestUser()
├── shared/
|   ├── errors/__tests__/     ← AppError.test.ts (9)
|   ├── utils/__tests__/      ← utils.test.ts (12)
|   └── middlewares/__tests__/ ← authenticate (7), validate (5), errorHandler (4)
└── modules/
    ├── auth/__tests__/       ← dto (16), service (13)
    ├── contacts/__tests__/   ← dto (11), service (13)
    ├── campaigns/__tests__/  ← dto (12), service (14)
    ├── interactions/__tests__/← dto (9), service (10)
    ├── dashboard/__tests__/  ← service (5)
    ├── ai/__tests__/         ← dto (9), prompt (10), service (8)
    └── messaging/__tests__/  ← dto (6), service (6)

frontend/src/
├── __tests__/setup.ts
├── api/__tests__/            ← client.test.ts (9)
├── utils/__tests__/          ← date.test.ts (6)
├── composables/__tests__/    ← useNotifications.test.ts (5)
└── stores/__tests__/         ← auth.store.test.ts (7)
```

4. Infrastructure Setup

4.1 Backend Vitest Configuration

```
// backend/vitest.config.ts
export default defineConfig({
  test: {
    globals: true,          // no need to import describe/it/expect
    environment: "node",    // backend = Node, not browser
    root: "src",            // resolve from src/ so imports match app code
    setupFiles: ["./__tests__/setup.ts"],
    include: ["**/__tests__/**/*.test.ts"],
    testTimeout: 15_000,    // DB ops can be slow on first run
    hookTimeout: 15_000,
    pool: "forks",          // isolate test files in child processes
    fileParallelism: false, // ← critical: prevents DB deadlocks
  },
});
```

Why `fileParallelism: false`? All integration tests share the same PostgreSQL database. Running them in parallel causes deadlocks on `TRUNCATE CASCADE` and foreign key violations. Sequential file execution solves this while still allowing tests *within* a file to run fast.

Why `pool: "forks"`? Vitest's default thread pool shares memory, which causes issues with `pg` connection pools. Forked processes get their own connection pool instance.

4.2 Setup File

```
// backend/src/__tests__/setup.ts
process.env.NODE_ENV = "test";
import dotenv from "dotenv";
dotenv.config({ path: path.resolve(process.cwd(), ".env.test"), override: true }
```

This runs before every test file. It loads `.env.test` which points to `pr_ai_db_test` on port 5433 — a dedicated test database that gets truncated between suites.

4.3 Test Helpers

```
// cleanDatabase() — resets all 8 tables between tests
await pool.query(`
  TRUNCATE users, auth_sessions, contacts, campaigns,
           campaign_contacts, interactions, ai_suggestions, audit_events
  CASCADE
`);

// createTestUser() — inserts a user directly (bypasses service layer)
// Uses bcrypt with 4 rounds (fast) instead of production's 12
const hash = await bcrypt.hash(password, 4);
```

Why bypass the service layer for user creation? Service tests need a user to exist *before* the test starts. Using the auth service would test two things at once. The helper inserts directly via SQL, keeping test setup independent of the code being tested.

4.4 Frontend Vitest Configuration

```
// frontend/vitest.config.ts
export default defineConfig({
  plugins: [vue()],
  resolve: { alias: { '@': './src' } },
  test: {
    globals: true,
    environment: 'jsdom',    // simulates browser DOM
    include: ['src/**/__tests__/**/*.test.ts'],
    setupFiles: ['src/__tests__/setup.ts'],
  },
});
```

Frontend tests use `jsdom` to simulate browser APIs (DOM, localStorage, fetch).

5. Testing Patterns & Conventions

5.1 The Three Test Categories

Category	DB?	Mocks?	Speed	Example
DTO / Schema	No	No	~20ms	`contacts.dto.test.ts`
Service Integration	Yes (real)	External APIs only	~300ms/test	`contacts.service.test.ts`
Middleware Unit	No	Express req/res	~30ms	`authenticate.test.ts`

5.2 Integration Test Lifecycle

Every service test file follows this exact pattern:

```
beforeAll(async () => {
  // Run the full schema migration to ensure tables exist
  const schema = fs.readFileSync("../config/schema.sql", "utf-8");
  await pool.query(schema);
});

afterAll(() => pool.end());   // release DB connections

beforeEach(async () => {
  await cleanDatabase();       // TRUNCATE all tables
  const user = await createTestUser();
  userId = user.id;            // fresh user for every test
});
```

Why `beforeAll` with schema migration? The test database might be empty on first run. Running `schema.sql` is idempotent (`CREATE TABLE IF NOT EXISTS`), so it works whether the DB is fresh or already set up.

Why `beforeEach` with truncation? Each test starts from a clean slate. No test depends on data from another test. This prevents the classic "test passes alone, fails in suite" problem.

5.3 Mocking External Dependencies

For AI and email providers, we mock at the module level:

```
// Mock AI provider — no real API calls
vi.mock("../ai.provider", () => ({
  generateCompletion: vi.fn().mockResolvedValue({
    subject: "Test subject",
    body: "Test body",
  }),
  getActiveModel: vi.fn().mockReturnValue("mock-model"),
}));
```

The rule: We mock *external boundaries* (OpenAI, Resend, etc.) but never mock the database. The database is part of the system under test — mocking it would give false confidence.

5.4 Error Assertion Strategy

Due to a quirk in how `AppError` uses `Object.setPrototypeOf`, `instanceof` checks for subclasses break under Vitest's module isolation. Instead of:

```
// ❌ This fails in Vitest even though it works at runtime
await expect(fn()).rejects.toThrow(NotFoundError);
```

We assert on the error message string:

```
// ✅ Reliable across all environments
await expect(fn()).rejects.toThrow("Contact not found");
```

This is actually *better* for documentation purposes — the test reads like a spec: "when you ask for a non-existent contact, you get 'Contact not found'."

6. Backend Tests — Module Breakdown

6.1 Shared Layer (37 tests)

AppError Hierarchy (9 tests)

Tests all error subclasses: `BadRequestError(400)`, `UnauthorizedError(401)`, `ForbiddenError(403)`, `NotFoundError(404)`, `ConflictError(409)`. Verifies status codes, messages, and that the base `AppError` class works correctly.

Utility Functions (12 tests)

- **`parseId`**: valid integers, zero, negatives, non-numeric strings, floats, arrays
- **`parsePagination`**: defaults (limit=50, offset=0), valid values, cap at 200, negatives

Validate Middleware (5 tests)

Tests the Zod validation middleware with valid body, missing fields, invalid email, error message formatting, and empty body.

Authenticate Middleware (7 tests)

Mocks `JWT_SECRET` via `vi.mock("config/env")`. Tests: valid JWT extraction → `req.userId`, missing Authorization header, non-Bearer scheme, expired token, wrong secret, malformed token, empty payload.

Error Handler Middleware (4 tests)

Tests the Express error handler: known errors (BadRequest → 400), NotFound → 404, unknown errors → 500, and custom AppError subclasses.

6.2 Auth Module (29 tests)

DTO Validation (16 tests)

- **Register**: valid data, short password (<8 chars), invalid email, missing fields
- **Login**: valid, missing email, missing password, invalid email format
- **Refresh/Logout**: valid token, missing token, empty string

Service Integration (13 tests)

Test	What It Verifies
Register (happy path)	Returns access + refresh tokens, user created in DB

Register (duplicate)	Throws "Email already registered"
Login (valid)	Returns tokens for correct credentials
Login (wrong password)	Throws "Invalid credentials"
Login (non-existent)	Throws "Invalid credentials" (same message — no email enumeration)
Refresh (rotation)	Issues new tokens, old refresh is invalidated
Refresh (reuse detection)	Old token rejected after rotation
Refresh (invalid)	Random string throws "Invalid refresh token"
Logout	Revokes session, token no longer works
Logout (idempotent)	Revoking an already-revoked token doesn't throw
Logout all	All sessions for user are invalidated
Me (valid)	Returns user profile
Me (non-existent)	Throws for deleted user

> **Security note:** Login errors use the same message for wrong password and non-existent email. This prevents email enumeration attacks.

6.3 Contacts Module (24 tests)

DTO Validation (11 tests)

- **Create**: valid, minimal (defaults for outlet/topics), missing name, invalid email, empty name, name >255 chars
- **Update**: partial updates, empty object, archivedAt as null (restore), archivedAt as ISO string, invalid email

Service Integration (13 tests)

Test	What It Verifies
Create	Returns full DTO with score=0 and archivedAt=null
Get	Retrieves by ID
Get (not found)	Throws "Contact not found"

Get (wrong user)	User A can't see User B's contacts
List	Paginated results with total count
List (excludes archived)	Soft-deleted contacts don't appear
List (pagination)	Limit/offset work correctly
Update	Partial field updates, unchanged fields preserved
Update (not found)	Throws for non-existent
Update (archived)	Throws "Cannot update an archived contact. Restore it first."
Delete (soft)	Sets `archived_at`, contact still retrievable
Delete (not found)	Throws for non-existent
Delete (already archived)	Throws — can't archive twice

6.4 Campaigns Module (26 tests)

DTO Validation (12 tests)

- **Create**: valid, minimal, empty name, name >255, description >2000
- **Update**: partial, valid statuses (draft/active/paused/completed), invalid status
- **AddContacts**: valid array, empty array, non-positive IDs, non-integers

Service Integration (14 tests)

Test	What It Verifies
Create	Returns campaign with status="draft"
Get / Get (not found)	Standard retrieval + error
List / List (excludes archived)	Pagination + soft delete filtering
Update	Name + status change
Add contacts	Bulk insert via `campaign_contacts` junction
Add contacts (duplicate)	`ON CONFLICT DO NOTHING` returns added=0
List participants	Returns contact details via JOIN
Remove contact	Deletes from junction table

Add contacts (wrong user)	Throws when contacts belong to another user
Delete (soft)	Campaign still retrievable but archived

6.5 Interactions Module (19 tests)

DTO Validation (9 tests)

- Valid directions: `inbound`, `outbound`, `internal`
- Valid channels: `email`, `note`
- Valid statuses: `draft`, `sent`, `delivered`, `failed`, `replied`, `archived`
- Full data with all optional fields (campaignId, subject, occurredAt, metadata)

Service Integration (10 tests)

Test	What It Verifies
Create outbound	Interaction created + `contact.last_contacted_at` updated
Create inbound	Direction and status correct
Create note	Internal channel works
Create (no contact)	Throws "Contact not found"
Create (archived contact)	Throws "Cannot create interaction for an archived contact"
Create (score update)	Relationship score recalculated after interaction
Get / Get (not found)	Standard retrieval
List by contact	Filtered + paginated
List all	All user interactions

> **Why score recalculation matters:** The relationship score is computed from interaction history. Creating an outbound email should increase the score. This test verifies the transactional side effect.

6.6 Dashboard Module (5 tests)

Test	What It Verifies
Zero stats	New user gets all zeros, responseRate=0 (not NaN)
Counts	totalContacts and activeCampaigns computed from DB
Archived excluded	Soft-deleted contacts not counted

Recent activity	Audit events mapped to activity feed items
Empty activity	No audit events → empty array

6.7 AI Module (27 tests)

DTO Validation (9 tests)

- **Generate**: defaults (tone=neutral, length=medium), custom values, invalid tone/length, missing IDs
- **Reject**: optional reason, reason >500 chars

Prompt Builder (10 tests)

Tests `buildPrompts()` which constructs the system and user prompts sent to OpenAI/Gemini:

- Contact info included in user prompt (name, outlet, topics)
- Campaign objective included
- Recent interactions summarized
- Empty interactions → "No previous interactions"
- Tone instructions (warm → "friendly", direct → "concise")
- Length constraints (short → "under 100 words", long → "200–350 words")
- JSON output format required in system prompt

Service Integration (8 tests)

AI provider is mocked — no real API calls.

Test	What It Verifies
Generate	Calls provider, stores suggestion in DB, returns subject/body/model
Generate (no contact)	Throws "Contact not found"
Generate (no campaign)	Throws "Campaign not found"
Generate (archived)	Throws "Cannot generate outreach for an archived contact"
Accept suggestion	Status transitions to "accepted"
Reject suggestion	Status transitions to "rejected" with optional reason
Accept (already processed)	Throws — can't accept twice
List suggestions	Paginated results

6.8 Messaging Module (12 tests)

DTO Validation (6 tests)

- Valid data, optional campaignId/aiSuggestionId, empty subject/body, missing contactId, subject >500

Service Integration (6 tests)

Both AI provider and email provider are mocked.

Test	What It Verifies
Send	Creates interaction + returns providerMessageId + status="sent"
Send (updates contact)	`last_contacted_at` is updated
Send (no contact)	Throws "Contact not found"
Send (archived)	Throws "Cannot send email to an archived contact"
Send (with AI suggestion)	Marks suggestion as "sent" after successful delivery
Send (non-accepted suggestion)	Throws "Cannot send: suggestion status is 'draft', expected 'accepted'"

7. Frontend Tests — Module Breakdown

7.1 API Client (9 tests)

Tests the custom fetch wrapper in `api/client.ts`:

Test	What It Verifies
setTokens / clearTokens	localStorage persistence
loadTokens	Hydrates from storage on app init
hasRefreshToken	Boolean check for token presence
ApiError class	Custom error with status code
GET with auth	Authorization header attached automatically
POST with body	JSON serialization, Content-Type header
Error response	Throws `ApiError` with server message
204 No Content	Returns null (no body parsing)

7.2 Date Utilities (6 tests)

Uses `vi.useFakeTimers()` to freeze time at a known date:

- `formatDate`: Locale-aware date formatting
- `formatRelativeDate`: "2 hours ago", "3 days ago", "just now" — tests various time deltas

7.3 Notifications Composable (5 tests)

Tests `useNotifications()` — a reactive notification queue:

Test	What It Verifies
notify()	Adds notification with unique ID
remove()	Removes by ID
Auto-dismiss (success)	Disappears after 4 seconds (fake timers)
Auto-dismiss (error)	Disappears after 8 seconds

MAX_VISIBLE cap	Queue limits prevent UI overflow

7.4 Auth Store (7 tests)

Tests the Pinia auth store with mocked API layer:

Test	What It Verifies
Initial state	user=null, isAuthenticated=false
Register	Calls API, stores tokens, sets user
Login	Same flow as register
Logout	Calls API, clears tokens, resets state
tryRestore (no token)	Skips restoration silently
tryRestore (valid)	Refreshes tokens, loads user profile
tryRestore (expired)	Clears stale tokens, stays logged out

8. Challenges & Solutions

8.1 `happy-dom` EPERM on Windows

Problem: `happy-dom` (an alternative to jsdom) caused persistent `EPERM` file locking errors on Windows. The `.pnpm-store` couldn't delete or overwrite its files, even after `pnpm store prune`.

Solution: Added a pnpm override in the root `package.json` to replace `happy-dom` with an empty package:

```
{
  "pnpm": {
    "overrides": {
      "happy-dom": "npm:@pnpm/empty-pkg@1.0.0"
    }
  }
}
```

Then used `jsdom` instead. Works identically for our test needs.

8.2 `instanceof` Broken for AppError Subclasses

Problem: `AppError` uses `Object.setPrototypeOf(this, AppError.prototype)` for ES5 compatibility. This breaks `instanceof` checks for subclasses (`NotFoundError` , etc.) when Vitest's module isolation creates separate copies of the class.

Solution: Assert on error messages instead of types. This turned out to be more readable anyway.

8.3 Database Deadlocks in Parallel Test Files

Problem: Integration test files running in parallel would deadlock on `TRUNCATE CASCADE` because multiple processes were trying to lock the same tables simultaneously.

Solution: Set `fileParallelism: false` in `vitest.config.ts` . Tests within a single file still run sequentially (as integration tests should), and files run one after another. Total suite time went from 42s (with failures) to 37s (all passing).

8.4 `vitest.config.ts` Outside `rootDir`

Problem: TypeScript complained that `vitest.config.ts` was outside the `src/` rootDir.

Solution: Added `"exclude": ["vitest.config.ts"]` to `backend/tsconfig.json` .

8.5 Setup File Double-Path

Problem: With `root: "src"` and `setupFiles: ["./src/__tests__/setup.ts"]`, Vitest resolved to `src/src/__tests__/setup.ts`.

Solution: Changed to `./__tests__/setup.ts` (relative to the root, which is already `src/`).

---

9. Running the Tests

```
# Backend — all 179 tests
cd backend && pnpm test

# Backend — watch mode (re-runs on file changes)
cd backend && pnpm test:watch

# Backend — with coverage report
cd backend && pnpm test:coverage

# Frontend — all 27 tests
cd frontend && pnpm test

# Frontend — watch mode
cd frontend && pnpm test:watch

# Run everything from root
pnpm --filter @pr-outreach/backend test && pnpm --filter @pr-outreach/frontend t
```

Prerequisites:

- PostgreSQL running on port 5433 (via Docker: `docker compose up -d` in `backend/`)
- Test database `pr_ai_db_test` must exist
- `backend/.env.test` must be configured with the test DB connection string

---

10. Test Coverage Summary

Module	Unit Tests	Integration Tests	Total
Shared (errors, utils, middleware)	37	—	**37**
Auth	16	13	**29**

Contacts	11	13	**24**
Campaigns	12	14	**26**
Interactions	9	10	**19**
Dashboard	—	5	**5**
AI	19	8	**27**
Messaging	6	6	**12**
Backend subtotal	**110**	**69**	**179**
API Client	9	—	**9**
Date Utils	6	—	**6**
Notifications	5	—	**5**
Auth Store	7	—	**7**
Frontend subtotal	**27**	**—**	**27**
Grand total	**137**	**69**	**206**

---

11. What's Next

The current suite covers **validation, business logic, and state management**. The roadmap for future phases:

Phase	Focus	Status
Phase 1	Foundation + Auth	✅ Done
Phase 2	All backend services + DTOs	✅ Done
Phase 3	API integration tests (supertest)	Planned
Phase 5	Frontend component tests	Planned
Phase 6	E2E smoke tests (Playwright)	Planned
Phase 7	CI/CD pipeline integration	Planned

The priority for Phase 3 is testing the full HTTP request → middleware → service → DB → response cycle with `supertest`. This catches bugs that live in the controller layer (parameter parsing,

response formatting, status codes).

Built with care. Break things in tests, not in production.